P1389R0
Standing Document for SG20: Guidelines for Teaching C++ to Beginners

Draft Proposal, 2019-01-21

This version:
https://wg21.link/p1389

Authors:
Christopher Di Bella
Simon Brand
Michael Adams

Audience:
SG20

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

P1389 proposes that SG20 create a Standing Document for guidelines for teaching introductory C++, and a handful of proposed initial guidelines.

Table of Contents

§ 1. Motivation for a set of Teaching Guidelines for Beginners

§ 1.1. Who is a ‘beginner’?

The term ‘beginner’ in P1389 targets students who have previously written programs before, but have little-to-zero training in writing C++ programs.

§ 1.2. Why beginner guidelines?

Introducing C++ to beginners is a delicate task, and is how novices develop their first impression of the language. Novices should be guided, not by being presented with language features, but rather how to write programs using C++. P1389 very strongly advocates for avoiding teaching beginners low-level ‘things’ such as pointers, bit hacking, explicit memory management, raw arrays, threads, and so on, in the *early* stages of their development process. Similarly, beginners do not need to be aware of the twenty-or-so fundamental types from the get-go.

In order to prevent overwhelming novices-to-C++, P1389 requests that beginner guidelines recommend beginners be exposed to a subset of C++ that encourages designing and engineering programs using the lightweight abstractions that set C++ apart from other programming languages.

These guidelines are not necessarily meant to be considered in isolation. For example, Dan Saks has mentioned that introducing C++ to C programmers requires care in the first features that are introduced, and -- in his experience -- that starting with `std::vector` as a *replacement* for raw arrays early on is often counter-productive. P1389 does not propose a *C++ for C programmers* Standing Document, but recommends a later proposal do exactly that. Teachers designing curricula for introducing C++ to C programmers would then be encouraged to encouraged read *both* guidelines.

§ 2. Guidelines

Each of the following subsections is a proposed guideline.

§ 2.1. What are beginner topics?

We divide beginner topics into three stages. Each stage represents prerequisite knowledge for the next stage. The contents of a particular stage might be revised in later stages. For example, error handling is necessary in Stage 1, but the topic should be re-visited later on so that error handling is addressed in-depth.

Beyond the stage partitions, these lists are sorted alphabetically. Chronological sorting is intended to be a discussion point for SG20.

§ 2.1.1. Stage 1 (fundamentals)

- algorithms (sequential)
- basic I/O
- computation constructs in C++ (expressions, sequence, selection, iteration)
- `constexpr` variables
- containers
- contracts
- designing and using functions and lambdas
- designing and using classes
- error handling (e.g. exceptions through helper functions, error types such as `std::optional` and `std::expected`)
- implicit and explicit conversions
- include directive
- iterator usage (simply comparison to `end(r)` and `*i`)
- modules
- operator overloading
- program design
- ranges
- references
- scoped enumerations
- testing (see PYYYY)
- the C++ compilation model
- the C++ memory model
- the C++ execution model
- tooling (e.g. compiler, debugger, package manager)
- types, objects, variables, and `const`

Editor's note: Discussion about `constexpr` as a Stage 1 topic has happened between the author and multiple reviewers, suggesting that consensus is lacking on this topic. It is requested that the placement of `constexpr` be a discussion point for SG20.

2.1.2. Stage 2 (todo: name me)

- algorithms (parallel)
- benchmarking
- `constexpr` functions
- derived types (for interface reasons -- implementation reasons to be pushed to 'intermediate')
- exceptions
- run-time polymorphism
- scope
- smart pointer use
- using libraries

2.1.3. Stage 3 (todo: name me)

- free store (and why you should avoid its direct usage)
- iterator use beyond Stage 1
- generic programming (a *gentle* introduction; rigor saved for 'intermediate')
- `std::pair` and `std::tuple` (and why you should avoid them)
- RAII
- sum types (e.g. `std::variant`)
- templates (a *gentle* introduction; template metaprogramming strictly excluded)

2.1.4. Stage 4 (todo: name me)

- interoperability with C
- interoperability with older C++ projects
- `std::unordered_map` and `std::hash`

It is no accident that Stage 1 is significantly larger than Stages 2, 3, and 4 combined. A large portion of the contents of Stage 1 are chosen to help students develop both confidence in their use of C++ and a strong appreciation for designing and implementing programs using C++.

`std::unordered_map` is considered a Stage 4 topic solely because of the necessary template specialisations to have a custom type in the associative container. Students should be thoroughly comfortable with templates before they are specialising `std::hash`.

2.2. [types] Types

2.2.1. [types.basic] Basic types

C++ supports a great many built-in types. Depending on the C++ Standard being used, there are as many as twenty one fundamental types in C++: eight distinct integer types, at least six distinct character types (six in C++11 through C++17, seven in the C++20 WP), three distinct floating-point types, `bool`, `void`, and `std::nullptr_t`. Further, there are the compound types, which include arrays of objects, functions, possibly cv-qualified pointers, possibly cv-qualified lvalue references, and possibly cv-qualified rvalue references, which some consider to be basic types, because they are built-in types.

An informal survey of textbooks and university courses done by the author has shown that many resources immediately introduce all of the fundamental types sans `std::nullptr_t` and `char8_t`, and there are a nonzero amount that very quickly introduce raw arrays, pointers, and references.

2.2.1.1. [types.basic.primary] Primary types

C++ novices rarely -- if ever -- have any need for more than a handful of types. In order to reduce the cognitive load on beginners, avoid introducing more than one of each fundamental type, postpone references until there is a relevant use-case, and avoid raw arrays and pointers for as long as possible.

The table below recommends these as the primary types for beginners.

Abstract type	Pre-C++20 type	Post-C++20 type
Integer	`int`	`int`
Floating-point	`double`	`double`
Boolean	`bool`	`bool`
Character	`char`	`char8_t`
String	`std::string`	`std::u8string`

Sequence container	`std::vector`	`std::vector`
Associative container	`std::map`	`std::map`

The distinction between pre-C++20 and C++20 is simply the acknowldgement of UTF-8. This is not to suggest that students should be introduced to the details of UTF-8 any earlier, but rather to get the idea of UTF-8 support on their radar, so that when they need to care about locales, they won't need to shift from thinking about why `char` is insufficient in the current programming world: they can just start using what they are already familiar with.

§ *2.2.1.2. [types.basic.conversions] Conversions*

Although discouraged whenever possible, conversions in C++ are sometimes necessary, and we cannot completely insulate beginners from this. [types.conversions] recommends that beginners be introduced to safe conversions (such as promotions) and unsafe conversions (such as implicit narrowing conversions).

```
auto c = 'a';
auto i = 0;

i = c; // okay, promotion
c = i; // not okay, implicitly narrows

i = static_cast<int>(c); // okay, but superfluous
c = static_cast<int>(i); // okay, explicit narrowing
c = gsl::narrow_cast<int>(i); // better, explicit narrowing with a descript
```

§ **2.2.2. [types.const] Constness**

§ *2.2.2.1. [types.const.constexpr] Encourage `constexpr` values whenever it is possible*

`constexpr` is a good way to ensure that values remain constant, and *variables* that are `constexpr` are constant expressions*.

As a general rule, default to `constexpr` unless something can only be known at run-time. `vector` and `string` always require run-time knowledge, so they cannot be `const`.

*Recommending `constexpr` does not mean explaining what a constant expression is. This is a separate discussion. For now, we can say "known at compile time".

2.2.2.2. [types.const.const] Encourage `const` whenever you can't use `constexpr`

`const` lets us reason about our programs with security and helps us produce more declarative code. Rather than suggesting that `const` is applied when you know that a value won't (or can't) change, offer `const` as the default, and suggest students *remove* `const` when they encounter a reason to mutate the variable.

*Editor's note: [types.const.const] does **not** suggest introducing lambda-initialisation (IILE).*

Editor's note: [types.const.const] becomes more and more easy-to-use when ranges are incorporated into a program.

2.2.2.3. [types.const.mutable] Don't pan `mutable`

TODO

(should `mutable` even be a topic for beginners? probably a stage 3 topic?)

2.2.3. [types.monadic] Types with monadic interfaces

TODO (visit after [P0798] receives a verdict).

2.3. [delay] Delay features until there is a genuine use-case

[basic.types] explicitly recommends avoiding the introduction of most fundamental types early on, as there is no use-case. Similarly, raw arrays, pointers, and even references are not considered members of [basic.types], as students will not appreciate them.

2.3.1. [delay.references] References

The author has found multiple resources that introduce pointers or references in the following fashion:

```cpp
// replicated introduction, not from an actual source
int i = 0;
int& r = i;

std::cout << "i == " << i << "\n"
   "r == " << r << '\n';
i = 5;
std::cout << "i == " << i << "\n"
   "r == " << r << '\n';

r = -5;
std::cout << "i == " << i << "\n"
   "r == " << r << '\n';
```

The above code offers no context for why references are necessary: only that reading and modifying `r` is synonymous to reading and modifying `i`, respectively. Without a genuine use-case, references can make seem C++ look rather quirky! Instead, it is recommended that students be exposed to references in a practical fashion, such as when passing parameters to functions.

§ 2.3.2. [delay.pointers] Pointers

Given that pointers solve a similar problem to references in terms of indirection, they share what is mentioned in [delay.references]. While pointers are an important part of C++ programming, their use-cases have been severely diminished thanks to references and abstractions such as `vector` and `map`.

References should definitely precede pointers by quite some time. This simplifies the idea of using C++ by eliminating syntax that often isn't necessary. Kate Gregory expands on this idea in [Stop-Teaching-C].

§ 2.3.3. [delay.iterators] Iterators

Iterators are a fundamental part of the standard library, which means that they can't be avoided in the context of standard library usage. The suggested guideline is for initial iterator usage:

```
// find gets a result                          // result != end(images) asks
if (auto result = find(images, date, &image::date); result != end(images))
   // 'training wheels'; *result gets the image, but then we go back to ref
   // funky syntax beyond operator* as a 'get' function.
   auto const& read = *result;
   display(read);

   auto& read_write = *result;
   change_hue(read_write, hue);
   display(read_write)
}
// can't use result outside of the condition
```

There has been a comment on why `display(*result)` is not directly applied. The above guideline does two things:

1. Keeps students away from the quirky syntax of iterators. Default to references.
2. Gets students into the mindest that an iterator's `operator*` returns a reference.

§ 2.3.4. [delay.concept.definitions] Concept definitions

Designing a concept is a lot of work, and is arguably an advanced topic; the world's foremost experts on the topic have stated that designing effective concepts comes after one has studied the details of algorithms. Even the definition for `EqualityComparable` is much more than just checking that `a == b` and `a != b` are syntactically possible.

This recommendation does not preculude the introduction of *using* existing concepts.

§ 2.3.5. [delay.cpp] C Preprocessor

Excludes `#include`, which is necessary until modules are in C++.

TODO

2.4. [style] Style practices

2.4.1. [style.guide] Use a style guide

- Recommended style guide: C++ Core Guidelines.
- See also: Use a code formatter
- See also: Use a linter

TODO (why?)

2.4.2. [style.naming] Use consistent set of naming conventions for identifiers

(e.g., names of variables, types, etc.)

To whatever extent is possible, a consistent set of naming conventions for identifiers should be employed. This practice helps to greatly improve the readability of code, amongst other things. Many popular naming conventions exist, and there are likely equally many opinions as to which one is best. Therefore, no attempt is made to advocate a particular one here. For examples of naming conventions that could be used, please refer to some of the popular style guides.

2.4.3. [style.`ALL_CAPS`] Avoid `ALL_CAPS` names

The use of `ALL_CAPS` is commonly reserved for macros. Developer tools, such as compilers and IDEs are able to quickly detect when a programmer is trying to write to something that is read-only (e.g. a constant).

Associated Core Guidelines:

- Enum.5: Don’t use ALL_CAPS for enumerators
- ES.9: Avoid ALL_CAPS names
- ES.32: Use ALL_CAPS for all macro names
- NL.9: Use ALL_CAPS for macro names only

Editor’s note: *Due to the lack of consensus, no other naming guidelines are made for variable or type names.* `ALL_CAPS` *are the exception because there appears to be a large enough consensus across multiple well-known style guides (Core Guidelines, Google Style Guide, and Mozilla Coding Style).*

§ 2.5. [projects] Projects

TODO (what?, why?, how?, where?, when?, how many?)

§ 2.6. [tools] Tools

Students should be introduced to a variety of basic tools for code development relatively early in the learning process (not later as an afterthought). The effective use of tools is important because this can make many tasks much easier, from formatting source code to testing and debugging. Not introducing at least some basic tools to the student will make their programming experience unnecessarily difficult and discourage the student from learning.

§ 2.6.1. [tools.compilers] Use an up-to-date compiler

The C++ language and standard library have been evolving rapidly in recent years. In order to ensure that newer language and library features are available, an up-to-date compiler is essential. Even if all of the latest language/library features are not needed for a course, using an up-to-date compiler is important for another reason. In particular, modern compilers have significantly improved error messages, making it easier for novices to find and correct their errors.

At the time of this writing, the most recent versions of several popular compilers are as follows:

- GCC: version 8
- Clang: version 7
- MSVC: version 2017 (with updates)

§ *2.6.1.1. [tools.multiple.compilers] Use two or more competing compilers*

No compiler is perfect. Some provide better diagnostics for certain types of problems than others. Giving the student the ability to use more than one compiler can be helpful when the error message from one compiler is not as enlightening as one would like. Also, some tools may only be available for a particular compiler. Therefore, in order to best utilize various tools, it is helpful for the student to be comfortable using more than one compiler.

§ 2.6.2. [tools.compiler.warnings] Use a high level of warnings and enable 'warnings as error' mode

Students should be taught to understand that the compiler is their friend. It can catch many problems in their code. Compiler warnings are one way in which the compiler can help the student to find problems in their code (such as a function with a missing return statement).

For example, one might use the following compiler flags:

- Minimum for GCC and Clang: `-Wall -Wextra -pedantic -Werror`
- Minimum for MSVC: `/W3 /WX`

§ 2.6.3. [tools.testing] Introduce a testing framework

Examples: Catch2, Google Test

Testing code is often viewed as tedious and boring by students, which discourages students from investing the time to properly test code. By using a testing framework, some of the monotony of testing can be reduced by eliminating the need for students to repeat boilerplate code that would be automatically provided by a test framework. By making testing less tedious to perform, students will be more motivated to do it well. Moreover, if a test framework that is popular in industry is chosen for teaching purposes, students will be further motivated by the knowledge that they are learning a useful tool in addition to developing their testing skills.

§ 2.6.4. [tools.debugger] Introduce a debugger early

Examples: Visual Studio Debugger, GDB, LLDB

The ability to step through running code and examine execution state will enable students to troubleshoot issues and correct assumptions they have made about the behavior of language and library features.

§ 2.6.5. [tools.package.management] Use a package manager

Examples: Vcpkg, Conan

Downloading, installing, and building against dependencies in C++ can be a challenge, especially for beginners. Package managers help alleviate this by providing tested packages along with automatic installation scripts.

§ 2.6.6. [tools.build.system] Use a build system

Example: CMake, Meson, build2

Build systems greatly aid building code across a variety of platforms. Without a build system, you will either require:

1. A uniform development environment for all students
2. Build instructions across a variety of supported environments, accounting for dependency installation locations, compiler, toolchain version, etc.

Neither of these are great solutions: you either need to ensure that all students have the necessary hardware and software to support the canonical environment and provide support for it, or you need to do a considerable amount of work to produce the necessary build instructions. Just use a build system.

§ 2.6.7. [tools.online.compiler] Introduce online compilers

Examples:

- Compiler Explorer
- Wandbox
- Coliru
- C++ Insights

Online compilers are invaluable tools for communicating about small snippets of code. Depending on the tool, they let the user compile programs using multiple toolchains, check the output of their code, and share the snippets with others.

Compiler Explorer's live updates can be particularly useful when experimenting with new features. The assembly view could overwhelm students however, so care should be taken when introducing this tool.

C++ Insights is a source-code transformation tool that can be particularly useful for helping the student to understand how the compiler views various types of code constructs in the language. For example, source code containing a lambda expression can be transformed by the tool into new (equivalent) source code that shows the closure type generated by the lambda expression. Many other code constructs are also handled by the tool (such as range-based for loops and structured bindings).

§ 2.6.8. [tools.code.formatter] Use a code formatter

Examples: Clang Format

Choosing a code formatter and picking a canonical style (it doesn't really matter which one) will avoid some code style arguments and improve uniformity among student's code. The latter will make marking and comparing solutions easier.

§ 2.6.9. [tools.linter] Use linters

Example: Clang Tidy

Static analysis tools are extremely useful for finding certain types of bugs or other problems in code. Students should be introduced to at least some basic static analysis tools (such as linters, like Clang Tidy) in order to instill the basic principle of finding bugs early (i.e., at compile time).

§ 2.6.10. [tools.runtime.analysis] Use run-time analysers, especially when teaching free store

Examples: Address Sanitizer (ASan), Undefined Behavior Sanitizer (UBSan)

Dynamic analysis tools can greatly improve the rigor with which code can be tested and also help to isolate bugs more quickly. Student should be introduced to basic dynamic analysis tools (such as ASan and UBSan) as such tools will help the student to more easily find problems in their code and also perhaps teach them that code that appears to run correctly can still have serious bugs that can be caught by such tools.

Notes: WSL does not play nicely with ASan, but a Docker image running inside WSL does.

§ 2.7. [appreciation] Appreciation for C++

§ 2.7.1. [appreciation.history] History

Do not introduce historical aspects of C++ in the forefront of C++ education. This includes:

> "C++ was developed by Bjarne Stroustrup in 1983 at Bell Labs as an extension to C and was previously known as 'C with Classes'..."

-- paraphrased introduction to C++ from many textbooks and courses informally surveyed by the author.

> "In the past we used SFINAE, which looks like *this*, now we use concepts..."

> "`int x[] = {0, 1, 2, ...}` is called an array and is how you store a group of objects..."

> "`printf` is used to write to screen..."

-- *paraphrased introductions to topics the author has seen.*

C with Classes was the immediate predecessor to C++, not an alternative name. This kind of statement helps embed the idea that C++ is simply 'C plus more', which is detrimental to a beginner's development of C++. It also incorrectly captures C++'s essence, which is not merely an extension to C, but also a *fusion* of ideals from Simula[PPP][dne] to support high-level abstractions in a lightweight fashion. In the author's experience, prematurely and inaccurately capturing the history of C++ gets programmers experienced with C into the mindset that programs engineered using C++ should be written in the image of C programs, and those who lack experience with C thinking that knowledge of C is a prerequisite.

While there is a very long history of C in C++[dne], this is not beneficial to beginners up-front, and should be pushed to a later time when students are able to appreciate history without first being exposed to the error-prone ways of the past. C++ programmers will eventually need to work with older code (pre-C++17 code is abundant), or write code that has C interoperability, and thus developing an appreciation for C++'s history is imperative (sic).

P1389 makes the case for it not to be in the first handful of unit.

§ 2.7.2. [appreciation.irl] C++ in the Real World

C++ has a broad range of applications. A non-exhaustive list of domains that C++ is used in can be found below, a large portion of which are derived from[applications].

- embedded System on a Chips (e.g. Renesas' R-Car H3[rcar])
- financial systems (e.g. Morgan Stanley; Bloomberg[bloomberg]; IMC Financial Markets[imc])
- graphics programming (e.g. Adobe technologies [adobe]; Mentor Graphics [mentor])
- middleware solutions (Codeplay Software[codeplay]; id Tech 4[id4])
- operating systems (e.g. Windows[win32])
- scientific computing (e.g. CERN)

- space technologies (e.g. NASA's Mars Rovers, James Webb Telescope)
- video games (e.g. Creative Assembly)
- web-based technologies (e.g. Google, Facebook, Amazon)

It is recommended that teachers briefly introduce a domain to their students during each unit. This practice has helped to broaden student appreciation for the usage of C++ in industry. Embedding use-cases into classes to show "this has practical value in the real world" should be considered.

§ 2.8. [meta] Meta-guidelines

This section is **not** about metaprogramming, but rather about guidelines for teachers to their teaching processes.

§ 2.8.1. [meta.revision] Regularly revise the curriculum

This is a living document, and will often change. It is strongly advised that you revise your curriculum between sessions to ensure that it does not stagnate or become out-of-sync with these guidelines.

§ 3. Student outcomes

Upon completion, a student should be able to:

§ 3.1. Containers

TODO

§ 3.2. Algorithms and ranges

TODO

§ 3.3. Error handling

TODO

§ 3.4. Testing

See PYYYY for now.

§ 3.5. Tool outcomes

- invoke a compiler in debug and release modes
- understand why using multiple compilers is important for writing well-formed C++
- understand why a high level of warnings is important for writing well-reasoned C++
    - understand why enforcing 'warnings as errors' mode is an important step up from just warnings
- configure a project
    - add and remove targets using the build system (IDE leverage a good practice)
    - add and remove packages using a package manager
- utilise a debugger, including
    - setting, disabling, skipping, and deleting breakpoints
    - setting, disabling, skipping, and deleting watchpoints
    - continue
    - stepping over
    - stepping into
    - stepping out
    - inspecting a variable’s contents
    - TUI-mode, if a command-line debugger is being used
- navigate Compiler Explorer, including
    - adding source files
    - adding compilers
    - use conformance mode
    - use diff mode
- understand why using a code formatter is a 'best practice'
- understand why using a code linter is a 'best practice'

§ 4. Acknowledgements

I’d like to thank Gordon Brown, Bjarne Stroustrup, JC van Winkel, and Michael Wong for reviewing.

§ Appendix A: Resources for Students

§ Programming -- Principles and Practice Using C++

- **Author**: Bjarne Stroustrup
- Webpage
- Teacher notes and author advice

§ A Tour of C++

- **Author**: Bjarne Stroustrup
- Webpage

§ C++ Reference

- **Short review**: C++ Reference is an wiki that is maintained by multiple C++ experts, many of whom participate in the standardisation of C++. C++ Reference is a highly navigable reference, with an online compiler so that examples can be experimented with.
- Webpage

§ Appendix B: Resources for Teachers

§ Stop Teaching C

- **Author**: Kate Gregory
- Video

§ How to Teach C++ and Influence a Generation

- **Author**: Christopher Di Bella
- Video

§ The Design and Evolution of C++

- **Author**: Bjarne Stroustrup
- Webpage

§ History of Programming Languages II

- **Author**: Bjarne Stroustrup
- Paper

§ History of Programming Languages III

- **Author**: Bjarne Stroustrup
- Paper

§ Appendix C: Glossary

- **Session**: A compilation of sessions. For a week-long C++ course, this would refer to the entire course. In a single semester of university, it refers to the full semester. For a textbook, a ‘session’ refers to the book’s ‘edition’.
- **Unit**: A unit of teaching. In a week-long C++ course, this might refer to hours or days. In a single semester of university, it refers to one full week (includes lectures, tutorials, and labs). In a textbook, a session is a single chapter. *Editor’s note: the term **unit** was previously known as **session**. This has now been changed.*

§ References

§ Informative References

[ADOBE]

Adobe, Inc.. GitHub for Adobe, Inc.. URL: https://github.com/adobe?utf8=%E2%9C%93&q=&type=&language=c%2B%2B

[APPLICATIONS]

Bjarne Stroustrup. C++ Applications. URL: http://stroustrup.com/applications.html

[BLOOMBERG]

How Bloomberg is advancing C++ at scale. 2016-08-23. URL: https://www.bloomberg.com/professional/blog/bloomberg-advancing-c-scale/

[CODEPLAY]

Codeplay Software, Ltd.. Codeplay -- The Heterogeneous Systems Experts. URL: https://codeplay.com/

[DNE]

Bjarne Stroustrup. The Design and Evolution of C++. URL: http://stroustrup.com/dne.html

[ID4]

id Software. DOOM-3-BFG. URL: https://github.com/id-Software/DOOM-3-BFG

[IMC]

IMC Financial Markets. IMC Summer of Code 2016. 2016. URL: https://www.boost.org/community/imc_summer_of_code_2016.html

[MENTOR]

Mentor Graphics. Mentor. URL: https://www.mentor.com/

[P0798]

Simon Brand. Monadic operations for `std::optional`. URL: https://wg21.link/p0798

[PPP]

Bjarne Stroustrup. Programming -- Principles and Practice Using C++. URL: http://stroustrup.com/programming.html

[RCAR]

Renesas. R-Car. URL: https://www.renesas.com/eu/en/products/automotive/automotive-lsis/r-car.html

[Stop-Teaching-C]

Kate Gregory. CppCon 2015: Stop Teaching C. URL: https://youtu.be/YnWhqhNdYyk

[WIN32]

Ken Gregg. In which language is the Windows operating system written?. URL: https://qr.ae/TUnniF